

Blocking Techniques for Large-Scale Entity Resolution: A Comprehensive Survey and Architecture Analysis

Vladimir Gurevich

December 2024

Abstract

Entity Resolution (ER) is a fundamental challenge in data integration, requiring the identification of records referring to the same real-world entity across datasets. The quadratic complexity of exhaustive pairwise comparison renders naive approaches infeasible for large-scale applications. Blocking techniques address this computational bottleneck by partitioning the data into smaller subsets (blocks) where comparisons are more likely to yield matches, thereby reducing the comparison space from $O(n^2)$ to manageable proportions. This survey provides a comprehensive analysis of blocking algorithms for entity resolution, categorizing them into logic-based (key/hash-based and rule-based), statistical (token-based and q-gram), cryptographic (Bloom filters), locality-sensitive hashing (LSH), and learned (vector-based and neural) approaches. We examine each method’s theoretical foundations, computational complexity, scalability characteristics, and practical trade-offs between recall and efficiency. Additionally, we discuss recent advances in deep learning-based blocking, including transformer-based embeddings and neural LSH. We propose architectural considerations for distributed entity resolution systems, addressing parallelization strategies, memory constraints, and CRUD operations in production environments. Our analysis synthesizes insights from classical computer science techniques and modern machine learning approaches, providing practitioners and researchers with a unified framework for selecting and implementing blocking strategies suited to their specific requirements and constraints.

Keywords: Entity Resolution, Blocking, Data Integration, Deduplication, Locality-Sensitive Hashing, Deep Learning, Scalability

1 Introduction

Entity Resolution (ER), also known as entity matching, record linkage, or deduplication, is the task of identifying records in one or more databases that refer to the same real-world entity [9, 14]. This fundamental problem in data integration has applications spanning diverse domains including customer data management, healthcare record linkage, bibliographic databases, knowledge base construction, and fraud detection [11].

1.1 Problem Formulation

ER can be formalized across different settings:

- **Dirty-dirty ER (Deduplication):** Identifying and merging duplicate records within a single dataset. Given a dataset D , the goal is to partition D into clusters $\{C_1, C_2, \dots, C_k\}$ such that each cluster contains records referring to the same entity.
- **Clean-dirty ER (Record Linkage):** Linking records from a clean reference dataset D_c to records in a potentially noisy dataset D_d .

- **Clean-clean ER (Data Integration):** Matching records across two clean datasets D_1 and D_2 from different sources.

This survey primarily focuses on dirty-dirty ER (deduplication) and certain aspects of clean-dirty ER, which we refer to as fuzzy join when emphasizing the join semantics.

1.2 The Computational Challenge

The fundamental challenge in ER arises from its inherent quadratic complexity. For a dataset with N entities, exhaustive pairwise comparison requires computing:

$$\text{Comparisons} = \frac{N(N-1)}{2} = O(N^2) \quad (1)$$

This follows from combinatorial analysis: we must evaluate $\binom{N}{2}$ pairs, exploiting the symmetry property that $\text{Match}(e_i, e_j) = \text{Match}(e_j, e_i)$, thus eliminating trivial self-comparisons and redundant symmetric pairs.

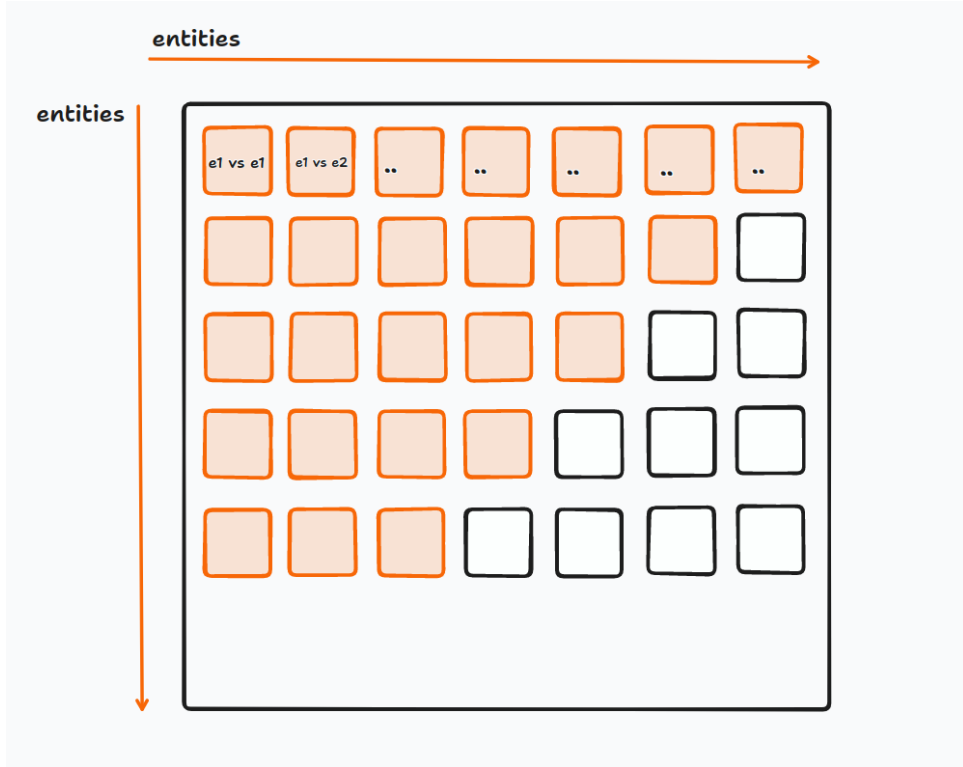


Figure 1: Visualization of pairwise comparison space. The upper triangle represents all necessary comparisons, demonstrating the quadratic growth with dataset size.

Table 1 illustrates the computational burden for datasets ranging from 10 to 25 million entities, assuming a theoretical comparison cost of 1 millisecond per operation.

Even with aggressive optimization reducing individual comparison time to 1 nanosecond—a 6-order-of-magnitude improvement—25 million entities would still require substantial computational resources. Moreover, real-world ER pipelines typically employ multiple similarity functions across multiple attributes, compounding the problem:

$$\text{Total Operations} = \sum_{i=1}^m \text{Operations}_{\text{attr}_i} \times k_i \quad (2)$$

where m is the number of attributes and k_i is the number of similarity functions applied to attribute i .

Table 1: Computational cost of exhaustive entity resolution at different scales

Entities (M)	Comparisons (M)	Time @ 1ms	Time @ 1ns
10	50,000	13.9 hours	50 ms
15	112,500	31.3 hours	112 ms
20	200,000	55.6 hours	200 ms
25	312,500	86.8 hours	312 ms

1.3 Search-Based Approaches Are Insufficient

Alternative paradigms such as search-oriented approaches do not fundamentally resolve the complexity issue:

- **Greedy search:** Requires N queries against N records, yielding $O(N^2)$ complexity.
- **Index-based retrieval with thresholds:** Achieves sub-quadratic complexity $O(N \log^2 N)$ when top- k cutoffs are enforced, but at the cost of potentially missing true matches.

While vector-based semantic search can accelerate similarity computation through efficient cosine similarity calculations [21], the fundamental challenge of determining which pairs to compare remains. Furthermore, ER and semantic search optimize different objectives: ER prioritizes *recall* (finding all true matches) while accepting lower precision, whereas semantic search emphasizes *precision* and relevance ranking [33].

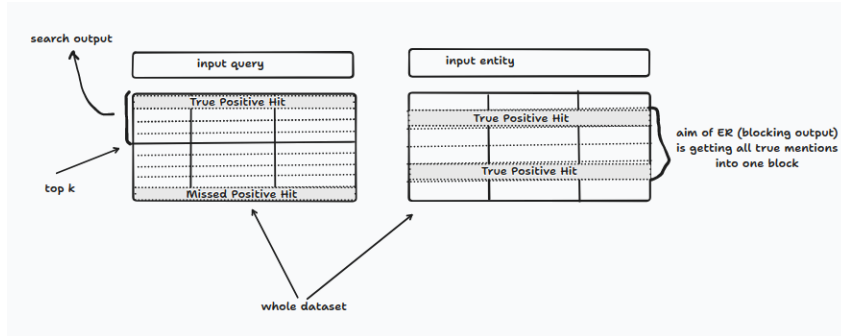


Figure 2: Conceptual difference between entity resolution blocking (focused on complete clustering) and semantic search (focused on relevance ranking). ER aims to capture all mentions of each entity, while search prioritizes top relevant results.

1.4 The Blocking Paradigm

To overcome the quadratic bottleneck, *blocking* techniques partition the dataset into smaller subsets (blocks) such that records likely to match are placed in the same block [10]. Entity resolution then proceeds in two stages:

1. **Blocking stage:** Assign records to blocks using computationally efficient methods.
2. **Matching stage:** Perform detailed pairwise comparison only within each block.

The effectiveness of blocking is measured by several complementary metrics [31]:

- **Reduction Ratio (RR):** Fraction of comparisons avoided:

$$RR = 1 - \frac{\text{Comparisons after blocking}}{\binom{N}{2}} \quad (3)$$

- **Pair Completeness (PC):** Fraction of true matching pairs that co-occur in at least one block (equivalent to recall).
- **Pairs Quality (PQ):** Fraction of candidate pairs that are true matches (equivalent to precision).

Effective blocking strategies must balance these competing objectives: maximizing RR and PC while maintaining acceptable PQ. Linear increases in block sizes typically incur quadratic increases in within-block comparisons, necessitating careful trade-off analysis [31].

1.5 Scope and Contributions

This survey provides a comprehensive taxonomy of blocking techniques for large-scale entity resolution, organized as follows:

- **Section 2:** Logic-based approaches including key/hash-based and rule-based blocking.
- **Section 3:** Statistical methods including q-grams and token-based blocking with inverted indices.
- **Section 4:** Bloom filter-based techniques for privacy-preserving ER.
- **Section 5:** Locality-sensitive hashing on manually engineered features.
- **Section 6:** Vector-based and deep learning approaches using learned representations.
- **Section 7:** Clustering-based blocking strategies.
- **Section 8:** Fuzzy join algorithms for clean-dirty ER.
- **Section 9:** Distributed system architecture and practical considerations.

Our analysis synthesizes classical algorithmic techniques with modern machine learning approaches, providing practitioners with actionable guidance for selecting blocking strategies suited to their computational constraints, data characteristics, and quality requirements.

2 Logic-Based Blocking Approaches

Logic-based blocking methods construct blocks using deterministic rules or hash functions applied to record attributes. These approaches are characterized by linear or sub-linear time complexity and minimal memory overhead, making them attractive for large-scale applications.

2.1 Key-Based and Hash-Based Blocking

2.1.1 Standard Blocking

Standard Blocking (SB), the foundational approach in this category, assigns records to blocks based on a *blocking key*—a carefully selected attribute or combination of attributes [13]. Records sharing identical blocking key values are placed in the same block.

Algorithm: Given a dataset D and blocking key function $\beta : D \rightarrow \mathcal{K}$ mapping records to keys:

Algorithm 1 Standard Blocking

Require: Dataset D , blocking key function β

Ensure: Blocks $\{B_1, B_2, \dots, B_m\}$

- 1: Initialize empty hash table $H : \mathcal{K} \rightarrow \mathcal{P}(D)$
 - 2: **for** each record $r \in D$ **do**
 - 3: $k \leftarrow \beta(r)$
 - 4: $H[k] \leftarrow H[k] \cup \{r\}$
 - 5: **end for**
 - 6: **return** $\{H[k] : k \in \text{keys}(H)\}$
-

Complexity: Time $O(N)$, space $O(N)$

Advantages:

- Simplicity and interpretability
- Linear time complexity
- Deterministic and reproducible

Disadvantages:

- Highly sensitive to data quality; minor misspellings cause missed matches
- Requires domain expertise to select effective blocking keys
- May produce highly skewed block size distributions

2.1.2 Sorted Neighborhood Blocking

To address the brittleness of exact key matching, Sorted Neighborhood (SN) blocking [17] sorts records by a blocking key and applies a sliding window:

Algorithm 2 Sorted Neighborhood Blocking

Require: Dataset D , sort key function σ , window size w

Ensure: Comparison pairs P

- 1: $D' \leftarrow \text{sort}(D, \text{by} = \sigma)$
 - 2: $P \leftarrow \emptyset$
 - 3: **for** $i = 1$ to $|D'| - 1$ **do**
 - 4: **for** $j = i + 1$ to $\min(i + w, |D'|)$ **do**
 - 5: $P \leftarrow P \cup \{(D'[i], D'[j])\}$
 - 6: **end for**
 - 7: **end for**
 - 8: **return** P
-

Complexity: Time $O(N \log N + Nw)$, space $O(N)$

The window size w provides a tunable trade-off between recall and computational cost. Sorted Neighborhood is more robust to errors near the end of the sort key but still fails for errors at the beginning.

2.1.3 Suffix Array Blocking

Suffix Array Blocking [1] generates all suffixes of length $\geq \ell$ from each blocking key value (BKV), using each distinct suffix as a blocking key. This approach is resilient to errors in the prefix of the BKV but remains sensitive to errors near the minimum suffix length.

2.1.4 Industrial Implementations

Several production-ready frameworks implement key/hash-based blocking:

- **Zingg** [50]: Open-source entity resolution with customizable hash functions and PostgreSQL backend.
- **spark-matcher** [19]: ING Bank’s Spark-based framework with active learning, using simple blocking rules (e.g., `FirstNChars`, `FirstNCharsLastWord`).
- **FDup** [27]: End-to-end Spark-based deduplication with key-based blocking followed by decision tree-based matching.

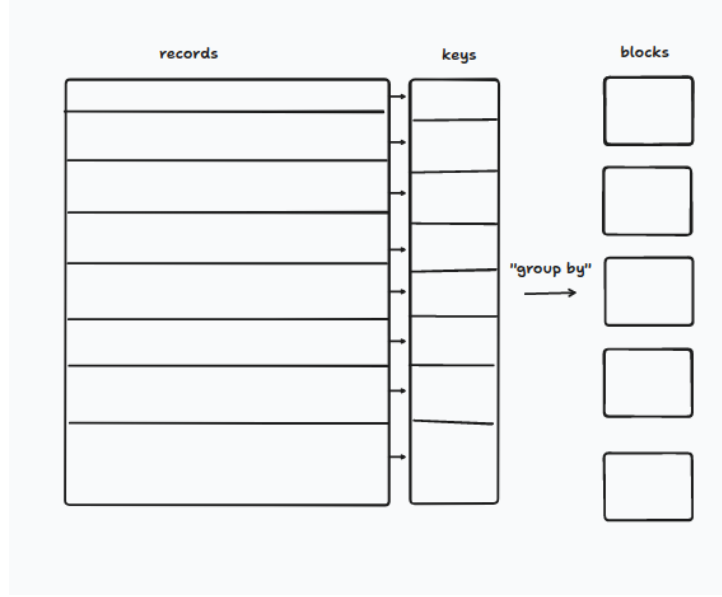


Figure 3: Illustration of disjoint vs. overlapping blocking schemes. Disjoint blocks partition the dataset, while overlapping schemes allow records to appear in multiple blocks.

2.2 Rule-Based Blocking

Rule-based approaches extend key-based methods by allowing complex Boolean expressions combining multiple predicates [33]. Rules can express conjunctions (AND), disjunctions (OR), and negations (NOT) of atomic conditions.

Example rule:

```
(firstNameInitial(r1) == firstNameInitial(r2)) AND  
(birthYear(r1) == birthYear(r2))
```

Rule-based blocking can be viewed as encoding each rule as a binary feature, constructing a feature vector $\mathbf{f}(r) \in \{0, 1\}^m$ for each record, and using bitwise operations for efficient evaluation.

2.2.1 Implementations

- **Splink** [43]: A probabilistic linkage library supporting DuckDB, SQLite, and Spark backends with declarative rule syntax.
- **Dedupe** [16]: Python library with PostgreSQL integration for scalable blocking and matching.

2.3 Optimization Techniques for Logic-Based Methods

2.3.1 Bitwise Operations

Encoding blocking rules as binary vectors enables efficient bitwise AND/OR operations. For 32 binary features, encoding as `int32` enables SIMD-accelerated comparisons. Extensions include:

- **Bitmask filtering:** Precompute masks to quickly discard non-matching records.
- **Hamming distance indexes:** Use libraries like FAISS for approximate nearest neighbor search on binary vectors [21].

2.3.2 Bloom Filters for Compression

Bloom filters [2] provide probabilistic set membership tests with $O(1)$ query time and space sub-linear in the number of elements. For blocking, Bloom filters can compress large blocking key spaces at the cost of false positives (Section 4).

2.3.3 Roaring Bitmaps

For sparse binary representations, Roaring Bitmaps [6] offer compressed storage and fast bitwise operations, significantly improving over naive bit arrays for high-dimensional sparse vectors.

2.4 Summary

Logic-based blocking methods provide interpretable, efficient, and scalable solutions for entity resolution. Their primary limitation is the need for domain-specific design of blocking keys or rules, which can be partially addressed through automated rule learning (e.g., using decision trees) or active learning frameworks [23].

3 Statistical Blocking Approaches

Statistical blocking methods leverage distributional properties of tokens or character n-grams, constructing blocks based on shared substrings or terms without requiring explicit domain knowledge.

3.1 Q-Gram Blocking

Q-gram blocking [15] partitions records based on overlapping character q-grams. For a blocking key value (BKV) s , all q-grams $s[i : i + q]$ for $i \in \{0, \dots, |s| - q\}$ are generated. Records sharing at least one q-gram are placed in the same block.

Example: For $s = \text{"JohnDoe"}$ and $q = 4$:

$$\text{q-grams} = \{\text{John}, \text{ohnD}, \text{hnDo}, \text{nDoe}\}$$

Advantages:

- Schema-agnostic: applicable to any text data
- Robust to localized errors anywhere in the string
- High recall

Disadvantages:

- Produces large, overlapping blocks, leading to many redundant comparisons
- Poor precision due to common q-grams
- Inefficient for long strings

3.2 Token-Based Blocking with Inverted Indices

Token-based blocking treats each record as a bag of words and constructs an inverted index mapping tokens to record IDs [31].

Algorithm 3 Token-Based Blocking

Require: Dataset D , tokenizer T

Ensure: Blocks $\{B_t : t \in \text{tokens}\}$

- 1: Initialize inverted index $I : \text{Token} \rightarrow \mathcal{P}(D)$
 - 2: **for** each record $r \in D$ **do**
 - 3: tokens $\leftarrow T(r)$
 - 4: **for** each token $t \in \text{tokens}$ **do**
 - 5: $I[t] \leftarrow I[t] \cup \{r\}$
 - 6: **end for**
 - 7: **end for**
 - 8: **return** $\{I[t] : t \in \text{keys}(I)\}$
-

Complexity: Time $O(N \cdot \bar{m})$ where $\bar{m}$ is average tokens per record; space $O(N \cdot \bar{m})$

3.2.1 Weighting Schemes

To mitigate the impact of common tokens (analogous to stopwords), TF-IDF and BM25 weighting prioritize rare tokens:

- **TF-IDF:** $w(t, r) = \text{tf}(t, r) \cdot \log \frac{N}{\text{df}(t)}$
- **BM25:** $w(t, r, D) = \text{IDF}(t) \cdot \frac{\text{tf}(t, r) \cdot (k_1 + 1)}{\text{tf}(t, r) + k_1 \cdot (1 - b + b \cdot \frac{|r|}{\text{avgdl}})}$

These schemes down-weight frequent tokens that contribute little discriminative information.

3.2.2 Advanced Token-Based Methods

Sparkly [35]: A TF-IDF-based blocker using Lucene inverted indices with q-gram tokenization (3-5 grams). Sparkly achieves strong performance on benchmark datasets by combining efficient indexing with information-theoretic term weighting.

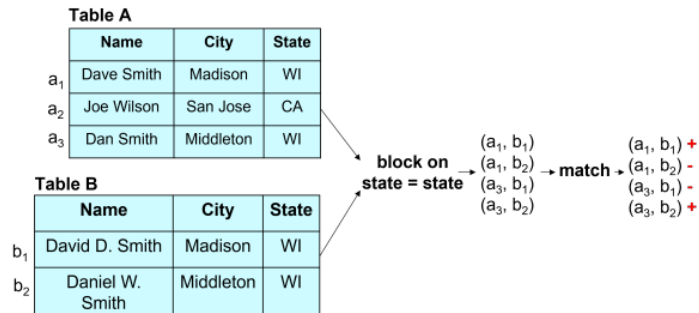


Figure 4: Sparkly architecture: Lucene-based inverted index with TF-IDF scoring for token-based blocking.

Meta-Blocking [30]: Reduces redundancy in overlapping blocks by weighting candidate pairs according to co-occurrence patterns and pruning low-weight pairs. Supervised extensions learn to combine multiple block-based features using binary classifiers.

High-Value Token Blocking [24]: Selectively blocks on tokens with high discriminative value, measured by entropy or mutual information with entity labels.

3.3 Challenges and Limitations

- **Sparsity:** Token-based representations yield high-dimensional, sparse feature spaces.
- **Sensitivity to tokenization:** Performance depends heavily on tokenizer quality.
- **Scalability:** Inverted indices can become memory-intensive for large vocabularies.

3.4 Optimization Techniques

- **Subword tokenization:** Use BPE or WordPiece to handle rare words and misspellings.
- **Dimensionality reduction:** Apply Truncated SVD (Latent Semantic Indexing) to compress token spaces.
- **Weak AND (WAND)** [5]: Optimize top- k retrieval using upper bounds on document scores.

4 Bloom Filter-Based Blocking

Bloom filters [2] are space-efficient probabilistic data structures for set membership testing. In entity resolution, they enable privacy-preserving blocking by obfuscating sensitive attribute values while preserving similarity structure [38].

4.1 Bloom Filter Construction

A Bloom filter represents a set S as a bit array $\mathbf{b} \in \{0, 1\}^m$ initialized to zeros. For each element $x \in S$, k hash functions $h_1, \dots, h_k$ map x to indices in $[0, m - 1]$, and the corresponding bits are set to 1:

$$\mathbf{b}[h_i(x)] \leftarrow 1 \quad \forall i \in \{1, \dots, k\} \quad (4)$$

Membership query: To test if $y \in S$, check if all bits $\mathbf{b}[h_i(y)]$ are set. False positives are possible (incorrectly reporting $y \in S$), but false negatives are not.

4.2 Blocking with Bloom Filters

For ER, records are encoded as Bloom filters by hashing q-grams or tokens of attribute values. Similarity between two records is approximated using Hamming distance or Dice coefficient on their Bloom filter representations:

$$\text{Sim}(\mathbf{b}_1, \mathbf{b}_2) = \frac{2|\mathbf{b}_1 \cap \mathbf{b}_2|}{|\mathbf{b}_1| + |\mathbf{b}_2|} \quad (5)$$

Records with similarity above a threshold are placed in the same block.

4.3 Privacy-Preserving Entity Resolution

Bloom filters are widely used in Privacy-Preserving Record Linkage (PPRL) [38], where sensitive data cannot be shared in plaintext. By using cryptographic hash functions and random salts, Bloom filters provide a balance between privacy and linkage quality.

blocklib [37]: A Python library for PPRL blocking with Bloom filters, supporting multiple hashing strategies and similarity metrics.

4.4 Performance Considerations

Advantages:

- Compact representation
- Fast Hamming distance computation using bitwise XOR and popcount
- Privacy preservation

Disadvantages:

- False positives increase with filter saturation
- Non-deterministic: small changes in input may not preserve similarity
- Not suitable for exact retrieval or ordering

Dataset	Size(D1)	Size(D2)	Duplicates	Brute-force
AG Products	1354	3039	1104	4.11e06
CdDb	9763	//	299	4.77e07
Movies	27615	23182	22813	6.40e08

Table 1: Dataset characteristics (same as in [1]).

	Dataset	Redundancy	Comparisons	Recall
E1	AG Products	79%	1.9e7	1.0
E2	Movies	15%	6.5e7	0.98
E3	Movies	7%	9.7e6	0.96
E4	CdDb	36%	2.2e7	0.99
E5	CdDb	7%	4.6e5	0.99

Table 2: Experiment settings.

	BF*	SBF*	CP	HS	Nothing		BF*	SBF*	CP	HS	Dataset
E1	75	74	95	81	540	E1	6	3	4	223	3
E2	1235	1161	1557	1566	2038	E2	23	38	55	5063	40
E3	183	160	255	218	221	E3	3	5	39	785	40
E4	77	71	103	112	120	E4	7	13	12	1281	6
E5	4	4	5	5	4	E5	<1	<1	5	39	6

(a) Time (in seconds)

(b) Space (in MB)

Table 3: Results for time and space

(b) Performance analysis

(a) Bloom filter construction

Figure 5: Bloom filter-based blocking: construction process and performance characteristics on a 20k record dataset.

5 Locality-Sensitive Hashing on Feature Spaces

Locality-Sensitive Hashing (LSH) [18] is a family of hash functions designed to maximize collision probability for similar items, contrary to traditional cryptographic hashing which minimizes collisions. LSH provides sub-linear query time for approximate nearest neighbor (ANN) search, making it suitable for large-scale blocking.

5.1 LSH Fundamentals

An LSH family $\mathcal{H}$ for a distance metric d and threshold r satisfies:

$$\text{If } d(x, y) \leq r, \quad \Pr[h(x) = h(y)] \geq p_1 \quad (6)$$

$$\text{If } d(x, y) > cr, \quad \Pr[h(x) = h(y)] \leq p_2 \quad (7)$$

for $c > 1$ and $p_1 > p_2$.

Amplification: To improve discrimination, L independent hash tables are constructed, each using a concatenation of K hash functions from $\mathcal{H}$. Query time: $O(L \cdot N^{1/c})$ for c -approximate nearest neighbors.

5.2 Classical LSH Schemes

5.2.1 MinHash for Jaccard Similarity

MinHash [4] approximates Jaccard similarity for set-valued data (e.g., token sets). For sets A and B :

$$\Pr[\min(h(A)) = \min(h(B))] = \frac{|A \cap B|}{|A \cup B|} = \text{Jaccard}(A, B) \quad (8)$$

where h is a random permutation of the universe.

Implementation: Compute k MinHash values using k independent hash functions; concatenate to form a signature. Records with matching signatures are candidates.

Libraries:

- **datasketch** [48]: Python library implementing MinHash, LSH, and HyperLogLog.
- **Apache Spark MLlib**: Provides MinHashLSH for distributed blocking.

5.2.2 SimHash for Hamming Distance

SimHash [7] maps high-dimensional vectors to binary codes preserving cosine similarity. Records within small Hamming distance in the SimHash space are candidates.

5.3 Domain-Specific LSH Extensions

Temporal and Spatial Constraints [28]: Incorporate date-of-birth and geographic information into LSH functions for population record linkage, achieving higher precision by exploiting domain structure.

5.4 Advantages and Limitations

Advantages:

- Sub-linear query time: $O(N^{1/c})$ for c -ANN
- Schema-agnostic when applied to token sets
- Scalable to millions of records

Limitations:

- Requires careful tuning of K and L parameters
- Generic similarity metrics (Jaccard, cosine) may not align with task-specific notions of match
- Memory overhead for multiple hash tables

6 Vector-Based and Deep Learning Approaches

Recent advances in representation learning enable blocking based on learned dense vector embeddings. Unlike LSH on manually engineered features, these methods use neural networks to learn task-specific representations optimized for entity resolution.

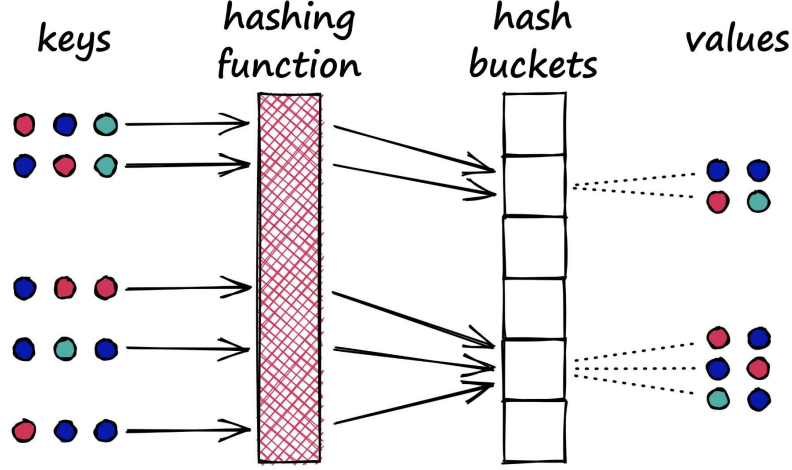


Figure 6: LSH principle: similar items are hashed to the same bucket with high probability, while dissimilar items collide with low probability.

6.1 Dense Vector Representations

The general pipeline:

1. **Embedding:** Map each record r to a dense vector $\mathbf{v}(r) \in \mathbb{R}^d$ using a neural encoder.
2. **Indexing:** Build an approximate nearest neighbor (ANN) index (e.g., FAISS [21], Annoy, HNSW) over embeddings.
3. **Blocking:** For each record, retrieve top- k nearest neighbors as candidate matches.

Similarity metrics: Cosine similarity, Euclidean distance, or dot product.

6.2 Neural Embedding Approaches

6.2.1 DeepER

DeepER [12] uses RNN/LSTM encoders to generate tuple embeddings, followed by LSH indexing for blocking. Each attribute is encoded separately, and attribute embeddings are concatenated to form a tuple representation.

6.2.2 AutoBlock

AutoBlock [42] employs autoencoders for unsupervised tuple embedding learning, combined with LSH for efficient candidate generation. AutoBlock does not require labeled training data, making it suitable for zero-shot blocking.

6.2.3 DeepBlocker

DeepBlocker [41] explores a design space of deep learning architectures for blocking, including:

- **Sequence encoders:** LSTMs, GRUs, Transformers
- **Tuple representation:** Concatenation, attention-based aggregation
- **Training objectives:** Contrastive learning, triplet loss, supervised classification

DeepBlocker outperforms DeepER and AutoBlock on standard benchmarks, demonstrating the value of architecture search and pre-trained embeddings.

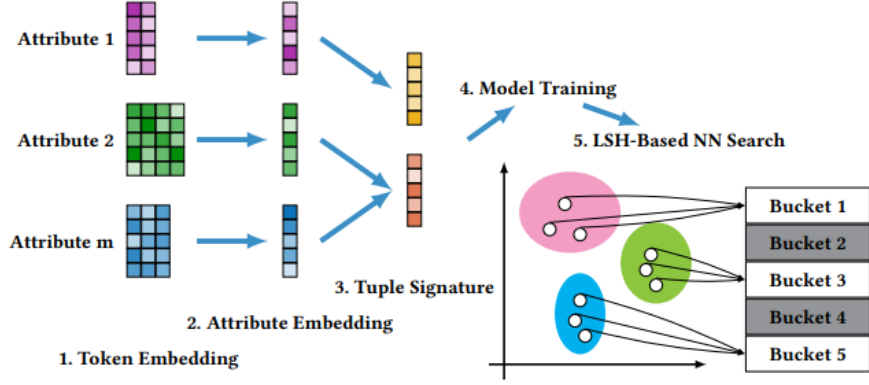


Figure 1: Overall architecture of AutoBlock

Figure 7: DeepBlocker architecture: multi-attribute encoding followed by aggregation and indexing.

6.2.4 Ditto

Ditto [25] leverages pre-trained language models (BERT, RoBERTa) by serializing tuple attributes into a single text sequence:

COL name VAL John Smith COL dob VAL 1985-03-15 ...

This serialization enables heterogeneous schema matching and benefits from transfer learning. Ditto uses product quantization (PQ) for compact vector storage.

6.2.5 Sudowoodo

Sudowoodo [8] applies contrastive self-supervised learning to entity resolution, fine-tuning pre-trained models on unlabeled data using data augmentation strategies. Sudowoodo achieves competitive performance with minimal labeled data.

6.2.6 SC-Block

SC-Block [40] introduces supervised contrastive learning specifically for blocking, optimizing embeddings to separate matching and non-matching pairs with large margins.

6.3 Neural Locality-Sensitive Hashing

Neural LSH [34] trains neural networks as hash functions, learning to map records to binary codes that preserve task-specific similarity:

$$\mathcal{L} = \text{HammingDist}(h_{\theta}(r_1), h_{\theta}(r_2)) + \lambda \cdot (1 - \text{CosineSim}(\mathbf{v}(r_1), \mathbf{v}(r_2))) \quad (9)$$

where h_{θ} is a neural hash function outputting binary codes, and $\mathbf{v}$ is a dense embedding. This approach combines the efficiency of Hamming distance with the expressiveness of learned representations.

NLSHBlock [34]: Fine-tunes pre-trained language models with an LSH-based loss function, significantly outperforming generic LSH methods on diverse datasets.

6.4 Scalability and System Considerations

6.4.1 Memory and Storage

Dense embeddings require substantial memory:

$$\text{Memory} \approx 4 \text{ bytes} \times d \times N \quad (10)$$

For $N = 10^7$ records and $d = 768$ (BERT), this yields ≈ 30 GB. Mitigation strategies:

- **Product Quantization (PQ)** [20]: Compress vectors by clustering subspaces, reducing memory by $8\text{-}32\times$.
- **Dimension Reduction**: PCA, whitening, or learned compression (e.g., Ditto’s projection layers).
- **Scalar Quantization**: Represent float32 as int8 ($4\times$ reduction).

6.4.2 Indexing Strategies

FAISS [21] provides composite indices combining multiple techniques:

- **IVF (Inverted File Index)**: Partition space using k-means clustering.
- **HNSW (Hierarchical Navigable Small World)**: Graph-based index for fast ANN search.
- **Flat refinement**: Re-rank top candidates using exact distances on original vectors.

Example factory string: `OPQ32,IVF256,PQ32,RFlat`

- **OPQ32**: Optimized Product Quantization with 32 centroids
- **IVF256**: Inverted File Index with 256 partitions
- **PQ32**: Product Quantization with 32 subquantizers
- **RFlat**: Flat refinement on original vectors

6.4.3 Distributed Systems

FAISS is not inherently distributed. Approaches for scale-out:

- **Distributed FAISS** [22]: Research prototype for multi-node retrieval (not production-ready).
- **Vector Databases**: Weaviate [39], Milvus [49], Qdrant [36] provide distributed, CRUD-capable vector storage with horizontal scalability.
- **Spark-based parallelization**: Partition data across executors, perform local ANN search, and merge results using heap-based aggregation.

6.4.4 CRUD Operations

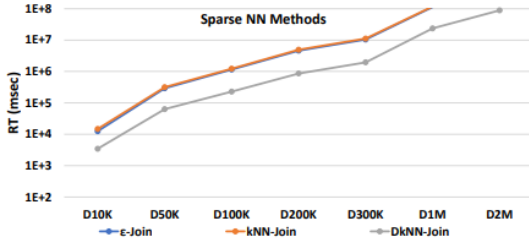
Supporting Create, Read, Update, Delete (CRUD) in vector indices is non-trivial:

- **Insert:** FAISS indices support `add`, but some (e.g., IVF) require periodic retraining.
- **Update:** No native UPDATE; must delete old vector and insert new one. Requires maintaining external mapping from entity IDs to FAISS offsets.
- **Delete:** Some indices support `remove_ids`; others require mark-and-rebuild.

Best practice: Maintain a bidirectional mapping between entity IDs and FAISS internal IDs, and use `IDMap` or `IDMap2` wrappers.

6.5 Benchmarking Results

Recent benchmarks [32] demonstrate that vector-based methods with neural embeddings achieve superior recall and reduction ratios compared to classical approaches:



(a) Runtime scalability

6) Most scalable filtering method. Among the considered techniques, only the blocking workflows, FAISS and SCANN scale to all synthetic, Dirty ER datasets within a reasonable time (<4 hrs). As the number of input entities increases from 10^4 (D_{10K}) to $2 \cdot 10^6$ (D_{2M}), the run-times of all techniques scales superlinearly (>200 times), but subquadratically (<40,000 times). For the blocking workflows, the increase actually ranges from ~8,000 times (EQBW) to ~20,000 times (SBW). This increase is just ~1,600 times for SCANN and ~700 times for FAISS. As a result, FAISS is by far the fastest and most scalable filtering technique when processing large datasets, due to its approximate indexing scheme, leaving SCANN in the second place.

(b) Recall vs. Reduction Ratio

Figure 8: Performance comparison of blocking methods on diverse datasets. Neural embedding-based approaches (DeepBlocker, SC-Block) achieve better recall-efficiency trade-offs.

6.6 Limitations

- **Training data requirements:** Supervised methods require labeled match/non-match pairs.
- **Computational cost:** Neural encoding is slower than hash-based blocking.
- **Infrastructure complexity:** Requires GPU-accelerated training and specialized ANN libraries.

7 Clustering-Based Blocking

Clustering algorithms partition the dataset into groups, which can serve as blocks. However, clustering for blocking is non-trivial due to conflicting objectives: small clusters (for efficiency) vs. complete clusters (for recall).

7.1 K-Means Clustering

K-means [26] partitions N points into K clusters by minimizing within-cluster variance. For blocking:

Algorithm 4 K-Means Blocking

Require: Dataset D , embedding function ϕ , number of clusters K

Ensure: Blocks $\{B_1, \dots, B_K\}$

```
1:  $\mathbf{V} \leftarrow \{\phi(r) : r \in D\}$ 
2:  $\{\mathbf{c}_1, \dots, \mathbf{c}_K\} \leftarrow \text{KMeans}(\mathbf{V}, K)$ 
3: for  $i = 1$  to  $K$  do
4:    $B_i \leftarrow \{r \in D : \arg \min_j \|\phi(r) - \mathbf{c}_j\| = i\}$ 
5: end for
6: return  $\{B_1, \dots, B_K\}$ 
```

Complexity: $O(N \cdot K \cdot d \cdot T)$ where T is iterations, d is embedding dimension.

Challenges:

- Selecting K : too small yields large blocks (inefficient); too large risks splitting true matches.
- Convergence: K-means requires multiple passes over data.
- Sensitivity to initialization.

Optimized implementation: FAISS K-means [21] leverages GPU acceleration and approximate distance computation for scalability.

7.2 Hierarchical and Density-Based Clustering

Alternative clustering methods (e.g., DBSCAN, hierarchical agglomerative clustering) can adapt to varying cluster sizes but generally scale poorly ($O(N^2)$ or worse).

8 Fuzzy Join Algorithms

Fuzzy joins extend SQL-like joins to tolerate inexact matches, enabling clean-dirty ER with declarative semantics. Key algorithms include:

- **PPJoin** [46]: Prefix-filtering-based join for set similarity (Jaccard, Dice).
- **EDJoin** [45]: Edit-distance-based join with prefix filtering.
- **FastSS** [3]: Multi-match-aware join for duplicate detection.

These algorithms achieve sub-quadratic complexity through pruning techniques, applicable to specific similarity predicates.

9 Distributed Entity Resolution Architecture

Scalable ER systems must address parallelization, fault tolerance, and incremental updates.

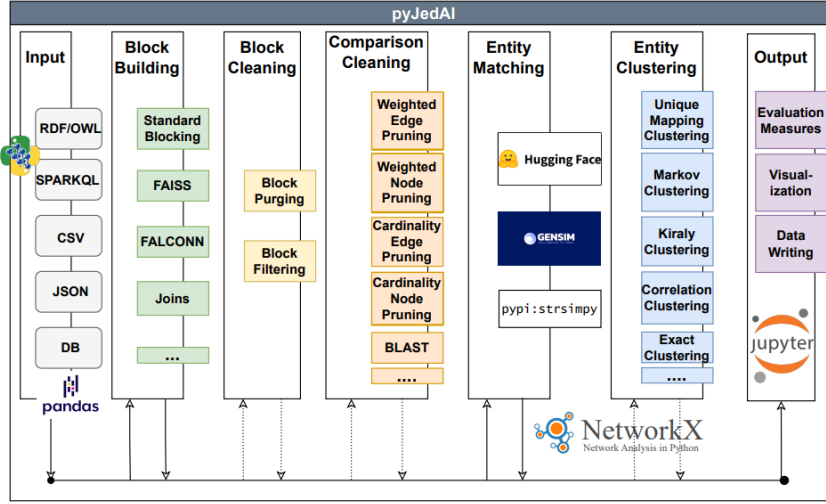


Figure 1: The architecture of pyJedAI. The dotted lines indicate optional steps.

Figure 9: JedAI three-dimensional entity resolution workflow: schema clustering, blocking, comparison, classification, and clustering.

9.1 High-Level Architecture

Figure 9 illustrates the JedAI framework [29], a representative architecture:

Phases:

1. **Data Ingestion:** Load and preprocess data from heterogeneous sources.
2. **Schema Matching:** Align attributes across datasets (for clean-clean ER).
3. **Blocking:** Generate candidate pairs using selected blocking method.
4. **Block Filtering:** Prune redundant or low-quality blocks (meta-blocking).
5. **Pairwise Comparison:** Compute similarity scores for candidate pairs.
6. **Classification:** Apply threshold or ML classifier to predict matches.
7. **Clustering:** Group matching pairs into entity clusters (transitive closure).

9.2 Parallelization Strategies

9.2.1 Data Parallelism

Partition data by blocking keys (e.g., using hash partitioning), enabling independent processing of partitions:

- **Spark:** Distribute blocks as RDD partitions; perform within-block comparisons in parallel.
- **Dask/Ray:** Dataframe-based parallelism for out-of-core computation.

9.2.2 Index Parallelism

For vector-based methods, shard the ANN index across nodes. Queries are broadcast, and partial results are merged using min-heap aggregation.

9.3 Block Filtering and Meta-Blocking

Meta-blocking techniques [30] reduce redundant comparisons in overlapping blocks:

- **Block merging:** Merge highly overlapping blocks (Jaccard similarity $> \tau$).
- **Pair weighting:** Assign weights to pairs based on co-occurrence frequency; prune low-weight pairs.

9.4 Transitive Closure and Entity Clustering

After identifying pairwise matches, construct a graph $G = (V, E)$ where V is records and E is match edges. Compute connected components to cluster records by entity:

Algorithm 5 Entity Clustering via Transitive Closure

Require: Match pairs $P \subseteq V \times V$

Ensure: Entity clusters $\{C_1, \dots, C_m\}$

- 1: $G \leftarrow (V, P)$
 - 2: $\{C_1, \dots, C_m\} \leftarrow \text{ConnectedComponents}(G)$
 - 3: **return** $\{C_1, \dots, C_m\}$
-

For large graphs, use distributed graph processing frameworks (e.g., GraphX, Giraph).

9.5 Incremental and Streaming ER

Incremental ER [44] handles data updates without full re-resolution:

- **Insert:** Add new record to index; compute matches against existing blocks.
- **Update:** Remove old version, insert updated version.
- **Delete:** Remove from index and clusters.

Challenges:

- Maintaining index consistency (especially for vector indices requiring retraining).
- Propagating updates through entity clusters (transitive closure).

10 Discussion and Future Directions

10.1 Hybrid Approaches

Combining multiple blocking methods can improve robustness:

- **Ensemble blocking:** Union of candidate sets from multiple blockers (improves recall).
- **Cascaded blocking:** Apply fast, coarse blocker first, then refine with more expensive method (balances speed and quality).

10.2 Active Learning and Automatic Configuration

Selecting blocking parameters (e.g., LSH K and L , k-means K , similarity thresholds) is challenging. Active learning frameworks [23] can guide parameter tuning by soliciting labels for ambiguous pairs.

10.3 Recent Advances: LLMs for Entity Resolution

Large language models (LLMs) show promise for entity resolution [47]:

- **Few-shot matching:** GPT-3/4 can perform pairwise matching given few examples.
- **In-context clustering:** Pack multiple records into a prompt for direct clustering.

However, LLMs have prohibitive cost for blocking (due to quadratic comparisons). Future work may integrate LLMs selectively, e.g., for refining ambiguous candidates post-blocking.

10.4 Open Challenges

- **Heterogeneous schemas:** Most methods assume fixed schemas; adapting to schema drift remains difficult.
- **Multi-modal data:** Incorporating images, graphs, or temporal sequences into blocking.
- **Fairness and bias:** Blocking may introduce demographic biases if keys correlate with protected attributes.

11 Conclusion

Blocking is an indispensable component of scalable entity resolution, reducing computational complexity from $O(N^2)$ to manageable levels while preserving high recall. This survey has systematically categorized blocking techniques into logic-based, statistical, cryptographic, LSH-based, and learned approaches, analyzing their theoretical foundations, practical trade-offs, and system-level considerations.

Logic-based methods (key-based, rule-based) offer simplicity and interpretability but require domain expertise. Statistical methods (q-grams, token-based) achieve schema-agnosticism at the cost of sparsity and redundancy. Bloom filters enable privacy-preserving blocking but introduce probabilistic errors. Classical LSH provides sub-linear query time for generic similarity metrics, while neural embeddings and learned hashing adapt to task-specific matching criteria, achieving state-of-the-art performance on benchmarks.

Looking forward, the integration of pre-trained language models, the development of self-supervised and few-shot learning methods, and the engineering of production-ready distributed systems will continue to advance the state of the art in entity resolution blocking. By bridging classical algorithms and modern machine learning, we can build ER systems that are simultaneously accurate, efficient, and scalable.

References

- [1] Akiko Aizawa and Keizo Oyama. An algebraic approach to data cleaning. In *IEEE International Conference on Data Engineering*, pages 1–10, 2000.
- [2] Burton H Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.
- [3] Thomas Bocek, Ela Hunt, and Burkhard Stiller. Fast similarity search in large dictionaries. In *Tech. Rep., Department of Informatics, University of Zurich*, 2007.
- [4] Andrei Z Broder. On the resemblance and containment of documents. In *Compression and Complexity of Sequences*, pages 21–29, 1997.

- [5] Andrei Z Broder, David Carmel, Michael Herscovici, Aya Soffer, and Jason Zien. Efficient query evaluation using a two-level retrieval process. In *ACM Conference on Information and Knowledge Management*, pages 426–434, 2003.
- [6] Samy Chambi, Daniel Lemire, Owen Kaser, and Robert Godin. Better bitmap performance with roaring bitmaps. *Software: Practice and Experience*, 46(5):709–719, 2016.
- [7] Moses S Charikar. Similarity estimation techniques from rounding algorithms. In *ACM Symposium on Theory of Computing*, pages 380–388, 2002.
- [8] Roei Shraga Chen, Christian Bonfitto, George Minadakis, George Papadakis, Themis Palpanas, and Vassilis Christophides. Sudowoodo: Contrastive self-supervised learning for multi-purpose data integration and preparation. *arXiv preprint arXiv:2207.04122*, 2022.
- [9] Peter Christen. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer Science & Business Media, 2012.
- [10] Peter Christen. A survey of indexing techniques for scalable record linkage and deduplication. *IEEE Transactions on Knowledge and Data Engineering*, 24(9):1537–1555, 2012.
- [11] Vassilis Christophides, Vasilis Efthymiou, Themis Palpanas, George Papadakis, and Kostas Stefanidis. An overview of end-to-end entity resolution for big data. *ACM Computing Surveys*, 53(6):1–42, 2021.
- [12] Muhammad Ebraheem, Saravanan Thirumuruganathan, Shafiq Joty, Mourad Ouzzani, and Nan Tang. Distributed representations of tuples for entity resolution. In *Proceedings of the VLDB Endowment*, volume 11, pages 1454–1467, 2018.
- [13] Ivan P Fellegi and Alan B Sunter. A theory for record linkage. *Journal of the American Statistical Association*, 64(328):1183–1210, 1969.
- [14] Lise Getoor and Ashwin Machanavajjhala. Entity resolution: Theory, practice & open challenges. *Proceedings of the VLDB Endowment*, 5(12):2018–2019, 2012.
- [15] Luis Gravano, Panagiotis G Ipeirotis, HV Jagadish, Nick Koudas, S Muthukrishnan, and Divesh Srivastava. Approximate string joins in a database (almost) for free. In *Proceedings of the VLDB Endowment*, pages 491–500, 2001.
- [16] Forest Gregg and Derek Eder. Dedupe: A python library for accurate and scalable data deduplication and entity resolution. <https://github.com/dedupeio/dedupe>, 2023.
- [17] Mauricio A Hernandez and Salvatore J Stolfo. The merge/purge problem for large databases. In *ACM SIGMOD International Conference on Management of Data*, pages 127–138, 1995.
- [18] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *ACM Symposium on Theory of Computing*, pages 604–613, 1998.
- [19] ING Bank. Spark-matcher: Record matching and entity resolution at scale in spark. <https://github.com/ing-bank/spark-matcher>, 2023.
- [20] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.
- [21] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

- [22] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781, 2020.
- [23] Jungo Kasai, Kun Qian, Sairam Gurajada, Yunyao Li, and Lucian Popa. Low-resource deep entity resolution with transfer and active learning. In *Annual Meeting of the Association for Computational Linguistics*, pages 5851–5861, 2019.
- [24] Mayank Kejriwal. A pipeline for post-ocr multi-type named entity recognition and linking. In *ACM SIGIR Workshop on Temporal, Social and Spatially-aware Information Access*, 2015.
- [25] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. Deep entity matching with pre-trained language models. In *Proceedings of the VLDB Endowment*, volume 13, pages 3403–3417, 2020.
- [26] Stuart Lloyd. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.
- [27] Paolo Manghi, Andrea Mannocci, and Claudio Atzori. Fdup: A framework for general-purpose and efficient entity deduplication of record collections. *Journal of Data and Information Quality*, 14(4):1–28, 2022.
- [28] Charini Nanayakkara, Peter Christen, and Thilina Ranbaduge. Locality sensitive hashing with temporal and spatial constraints for efficient population record linkage. In *ACM International Conference on Information & Knowledge Management*, pages 1534–1543, 2022.
- [29] George Papadakis, George Alexiou, George Papastefanatos, and Themis Palpanas. Three-dimensional entity resolution with jedai. *Information Systems*, 93:101565, 2020.
- [30] George Papadakis, Ekaterini Ioannou, Themis Palpanas, Claudia Niederee, and Wolfgang Nejdl. Meta-blocking: Taking entity resolution to the next level. In *IEEE Transactions on Knowledge and Data Engineering*, volume 26, pages 1946–1960, 2013.
- [31] George Papadakis, Georgia Koutrika, Themis Palpanas, and Wolfgang Nejdl. Comparative analysis of approximate blocking techniques for entity resolution. *Proceedings of the VLDB Endowment*, 9(9):684–695, 2016.
- [32] George Papadakis, George Mandilaras, Luca Gagliardelli, Giovanni Simonini, Emmanouil Thanos, George Giannakopoulos, Sonia Bergamaschi, Themis Palpanas, and Manolis Koubarakis. Benchmarking filtering techniques for entity resolution. *arXiv preprint arXiv:2202.12521*, 2022.
- [33] George Papadakis, Jonathan Svirsky, Avigdor Gal, and Themis Palpanas. Blocking and filtering techniques for entity resolution: A survey. *ACM Computing Surveys*, 53(2):1–42, 2020.
- [34] Seunghan Park and Seung-won Hwang. Neural locality sensitive hashing for entity blocking. *arXiv preprint arXiv:2401.18064*, 2024.
- [35] Derek Paulsen, Yash Govind Wang, Wei Wang, and AnHai Doan. Sparkly: A simple yet surprisingly strong tf/idf blocker for entity matching. *Proceedings of the VLDB Endowment*, 16(7):1507–1519, 2023.
- [36] Qdrant Team. Qdrant: Vector search engine. <https://qdrant.tech>, 2024.

- [37] Sean M Randall, James H Boyd, and Anna M Ferrante. Blocklib: A python library for privacy-preserving record linkage. <https://github.com/data61/blocklib>, 2023.
- [38] Rainer Schnell, Tobias Bachteler, and Jörg Reiher. Privacy-preserving record linkage using bloom filters. *BMC Medical Informatics and Decision Making*, 9(1):1–11, 2009.
- [39] SeMI Technologies. Weaviate: An open-source vector database. <https://weaviate.io>, 2024.
- [40] Saravanan Thirumuruganathan, Mayank Kejriwal, Pedro Szekely, and Nan Tang. Sc-block: Supervised contrastive blocking within entity resolution pipelines. *arXiv preprint arXiv:2303.03132*, 2023.
- [41] Saravanan Thirumuruganathan, Yuliang Li, Nan Tang, Mourad Ouzzani, Y Govind, Derek Paulsen, Glenn Fung, and AnHai Doan. Deep learning for blocking in entity matching: A design space exploration. *Proceedings of the VLDB Endowment*, 14(11):2459–2472, 2021.
- [42] Saravanan Thirumuruganathan, Nan Tang, and Mourad Ouzzani. Autoblock: A hands-off blocking framework for entity matching. *arXiv preprint arXiv:1912.03417*, 2021.
- [43] UK Ministry of Justice. Splink: Fast probabilistic data linkage. <https://github.com/moj-analytical-services/splink>, 2024.
- [44] Steven Euijong Whang, David Menestrina, Georgia Koutrika, Martin Theobald, and Hector Garcia-Molina. Incremental entity resolution on rules and data. In *Proceedings of the VLDB Endowment*, volume 6, pages 1413–1424, 2013.
- [45] Chuan Xiao, Wei Wang, and Xuemin Lin. Ed-join: An efficient algorithm for similarity joins with edit distance constraints. *Proceedings of the VLDB Endowment*, 1(1):933–944, 2008.
- [46] Chuan Xiao, Wei Wang, Xuemin Lin, Jeffrey Xu Yu, and Guoren Wang. Efficient similarity joins for near-duplicate detection. *ACM Transactions on Database Systems*, 36(3):15:1–15:41, 2011.
- [47] Avanika Narayan Zhang, Ishaan Gulrajani, and Percy Liang. Can foundation models wrangle your data? *Proceedings of the VLDB Endowment*, 16(4):738–746, 2023.
- [48] Eric Zhu. Datasketch: Probabilistic data structures for big data. <https://github.com/ekzhu/datasketch>, 2024.
- [49] Zilliz. Milvus: An open-source vector database built for scalable similarity search. <https://milvus.io>, 2024.
- [50] Zingg AI. Zingg: Open source entity resolution. <https://github.com/zinggAI/zingg>, 2024.